Hands-On-Practice: Diagnose and Fix Embedding Quality Issues

Description

Description: This activity immerses you in a realistic debugging scenario where an embedding-based recommendation system is producing poor results despite acceptable recall metrics. You'll apply visual diagnostics using UMAP to reveal hidden patterns in high-dimensional embedding space, then use DBSCAN clustering to systematically identify anomalous articles. Through metadata analysis and hypothesis formulation, you'll develop the investigative skills needed to trace embedding quality issues back to their data quality root causes and design actionable cleanup workflows that measurably improve system performance.

Learning Objectives

By the end of this activity, learners will be able to:

- Create UMAP 2D visualizations of high-dimensional embeddings color-coded by category labels.
- Apply DBSCAN clustering to systematically detect outlier articles that violate semantic coherence.

- Analyze metadata patterns to formulate testable hypotheses linking embedding anomalies to data quality issues.
- Design and prioritize data cleanup workflows that measurably improve embedding quality.

Assignment Components

Introductory Slide

Title: Data Scientist Mission: Fix News Aggregator's Bizarre Recommendations

Subtitle: Use UMAP and DBSCAN to diagnose why financial news appears with entertainment gossip

Tools to be used

- Python 3.9+ with pandas and NumPy
- UMAP library (umap-learn)
- DBSCAN from scikit-learn
- matplotlib and seaborn for visualization
- Jupyter Notebook for exploratory analysis

Scenario

You are a machine learning engineer at NewsAggregator, a content discovery platform that curates articles from 500+ publishers across 8 categories: Technology, Sports, Business, Entertainment, Health, Science, Politics, and Lifestyle. Recently, users have complained about bizarre article recommendations—sports articles appearing in technology feeds, entertainment news mixed with finance coverage. User engagement has dropped 18% this month, and your director suspects the embedding model is producing poor semantic representations due to data quality issues in the 10,000-article corpus.

Your task is to diagnose the root cause within 72 hours before the next board meeting. You've been given pre-generated embeddings (384 dimensions from all-MiniLM-L6-v2) for 10,000 recent articles, along with metadata including article_id, title, description, category, publish_date, and source publisher. Initial recall metrics show acceptable performance (71% recall@10), but the semantic coherence of recommendations is clearly broken.

Success means identifying at least 50+ outlier articles with specific data quality issues, formulating 3+ testable hypotheses about root causes supported by evidence, and proposing a prioritized cleanup plan that addresses high-impact problems first. Your recommendations will determine whether the company invests in data infrastructure improvements or abandons embedding-based personalization entirely.

Instructions for Learners

Step 1: Load Data and Create UMAP Visualization.

Download the article embeddings (10,000 articles × 384 dimensions) and metadata CSV.

Install UMAP: pip install umap-learn.

Load embeddings and metadata using pandas and NumPy.

Create UMAP reducer with n_neighbors=15, min_dist=0.1, n_components=2 (standard parameters).

Fit and transform embeddings to 2D: umap_coords = reducer.fit_transform(embeddings).

Create a scatter plot color-coded by article category using matplotlib or seaborn.

Save the visualization as high-resolution PNG or PDF for submission.

Step 2: Apply DBSCAN for Automated Outlier Detection

Import DBSCAN from sklearn cluster.

Apply DBSCAN to the UMAP coordinates with eps=0.5, min_samples=5 (tune if needed).

Identify outliers: articles with cluster label = -1.

Count total outliers and calculate outlier percentage: (num_outliers / 10000) * 100.

Cross-reference outliers with article categories to see if certain categories have higher outlier rates.

Export list of outlier article_ids to CSV for detailed analysis.

Step 3: Analyze Outlier Metadata Patterns

Sample 30-50 outlier articles and load their full metadata.

Check for missing or empty description fields—these produce poor embeddings.

Calculate word count for outlier descriptions: are they extremely short (<50 words)?

Identify if specific publishers contribute disproportionately to outliers (e.g., Publisher X has 40% of outliers).

Look for patterns in article titles: do they contain special characters, are multilingual, or lack context?

Examine categories: do certain categories (e.g., Entertainment) have legitimately diverse subtopics that appear scattered?

Step 4: Formulate and Document Hypotheses

Based on metadata analysis, formulate 3+ specific, testable hypotheses about root causes.

Example: "Hypothesis A: Articles from Publisher X have 3x higher outlier rate due to scraped content with poor metadata quality (evidence: 35 of 87 outliers from Publisher X)".

Example: "Hypothesis B: Articles with <100 words produce scattered embeddings due to insufficient semantic context (evidence: 60% of outliers have <75 words)".

For each hypothesis, cite specific quantitative evidence from your analysis.

Document hypotheses in a structured format (hypothesis statement + supporting evidence + proposed test).

Step 5: Design Prioritized Cleanup Workflow

Propose specific cleanup actions based on your hypotheses:

- Remove or re-scrape articles from problematic publishers.
- Implement minimum word count filter (100+ words) for future ingestion.
- Fill missing descriptions by extracting the first 200 characters from the article body.
- Split overly broad categories (e.g., Entertainment → Celebrity News, Movie Reviews).

Prioritize actions by estimated impact: fix Publisher X first if it accounts for 40% of outliers.

Define success metrics: target reducing outlier rate from X% to Y%, improving silhouette score from 0.4 to 0.65+

Propose validation plan: re-run UMAP after cleanup, measure cluster purity, A/B test recommendations with users.

Compile Your Findings

Requirement 1: UMAP Visualization

A 2D visualization revealing the semantic structure of article embeddings across all categories. Include: High-resolution scatter plot color-coded by article category, annotations highlighting problematic clustering patterns, and observations about category separation or unexpected overlaps.

Requirement 2: Outlier Analysis Report

A comprehensive analysis of articles identified as outliers by DBSCAN clustering. Include: Total outlier count and percentage, outlier distribution by category and publisher, metadata pattern analysis (word count, missing fields, source quality), and exported list of outlier article IDs.

Requirement 3: Hypothesis Documentation

Three or more data-driven hypotheses explaining the root causes of embedding quality issues. Include: Clear hypothesis statements, quantitative evidence supporting each hypothesis (specific numbers and percentages), proposed tests to validate each hypothesis, and connections between metadata patterns and embedding anomalies.

Requirement 4: Cleanup Plan and Validation Strategy

A prioritized workflow for addressing identified data quality issues. Include: Specific cleanup actions ranked by estimated impact, success metrics with target thresholds (outlier rate reduction, silhouette score improvement), validation methodology (re-run UMAP, cluster purity measurement), and A/B testing recommendations.

Summary

This activity provides hands-on experience with embedding quality diagnostics through practical application of dimensionality reduction visualization, automated outlier detection, and systematic root cause analysis. Participants have developed competency in creating UMAP visualizations for high-dimensional data interpretation, applying DBSCAN clustering for anomaly identification, and formulating evidence-based hypotheses that connect data quality issues to embedding performance, which are directly applicable to debugging production recommendation and search systems.

Submission Type	Recommended File Options
Written Work	Text, Audio (for verbal responses), Video (for sign language)
Visual Creation	Image, Video, Coding (for programmatic art)
Technical Solution	Coding, Text, Video (for demonstration)
Presentation	Video, Audio, Image (slides), Text (script)
Reflection	Text, Audio, Video

Share The Project

Document: Save the report in PDF format.

Upload: Share in the “Peer Review” area of the course shell with a brief description.

Instructions for Documenting and Sharing The Project for Peer Review

1. Document the Project

- Use word processing software to create your report.
- Ensure all sections of the project document structure are completed.
- Proofread and edit your report for clarity and accuracy.

2. Share the Project

- Save your report in PDF format.
- Upload your documents to the “Peer Review” area in the course shell.
- Provide a brief description of your project in the submission post.

Instructions for Documenting and Sharing The Project for Peer Review

3. Peer Review

- Review the projects submitted by your peers.
- Provide constructive feedback and comments on their work.